

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Jayavarapu Raju	Reg. No.:	192325117
Section / Batch:	Slot-A	Date:	

Code of Conduct

I J.Raju(192325117) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:J.raju

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & SyntaxDriven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

which is more suitable for large-scale applications.

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. CFG Trees vs Dependency Parsing

A **Context-Free Grammar (CFG)** represents a sentence as a hierarchical tree based on grammatical constituents such as noun phrases (NP), verb phrases (VP), and prepositional phrases (PP). For example, in “The student reads a book,” CFG groups “The student” as a noun phrase and “reads a book” as a verb phrase.

Dependency parsing represents the sentence through direct relationships between individual words. One word acts as the head, and other words depend on it. In “The student reads a book,” reads is the root, student is its subject, and book is its object.

Feature	CFG Trees	Dependency Parsing
Basic representation	Constituents / phrases	Word-to-word relationships
Main focus	Phrase structure	Grammatical dependencies
Structure	Hierarchical tree	Dependency tree
Subject/object relations	Indirect	Direct
Long-distance relationships	More difficult	Easier to represent
Useful for	Grammar checking and phrase analysis	Information extraction and semantic relationships

Justification: Dependency parsing is generally more effective for capturing relationships between words because it directly represents relations such as subject, object, modifier, and complement. CFG is better when the main requirement is constituent or phrase structure. Therefore, for applications that need relationships between words, dependency parsing is preferable.

2. Top-Down Parsing vs Earley Parsing

Top-down parsing starts with the start symbol of the grammar and tries to generate a structure that matches the input. **Earley parsing** is a chart-parsing algorithm that processes input while maintaining partial parsing states. It uses prediction, scanning, and completion.

Feature	Top-Down Parsing	Earley Parsing
Approach	Starts from start symbol	Maintains possible states in a chart
Backtracking	May require extensive backtracking	Reduces repeated work
Ambiguity	Can be inefficient	Handles ambiguity effectively
Incomplete input	Limited	Can maintain partial parses
Left recursion	Problematic in simple implementations	Can handle left-recursive grammars
Dynamic input	Less suitable	More suitable
Memory	Generally lower	Higher due to chart storage
Complex CFG	Less efficient	Well suited

For example, if the user enters “Book a flight to...”, a top-down parser may expect additional constituents and fail or repeatedly search for alternatives. Earley parsing can maintain the partial structure and continue processing when more words arrive.

Justification: For real-time systems involving incomplete, ambiguous, or dynamically arriving input, Earley parsing is more suitable. It avoids much of the repeated computation associated with backtracking and can maintain partial interpretations. Although it may require more memory, its flexibility makes it useful for voice assistants, conversational systems, and interactive NLP applications.

3. CFG vs PCFG vs Neural Parsing for Ambiguity

Consider: “**She saw the man with a telescope.**”

Interpretation 1: She used a telescope to see the man.

She → saw → man; “with a telescope” modifies the seeing event.

Interpretation 2: The man had a telescope.

She → saw → man; “with a telescope” modifies the man.

CFG: Can generate both valid parse trees, but normally has no mechanism to determine which interpretation is more likely.

PCFG: Assigns probabilities to grammar rules. If training data shows that one interpretation is more common, that parse can receive a higher probability and be selected.

Neural Parsing: Uses models such as transformers to learn contextual relationships from large datasets. It can use surrounding context to determine the most likely structure.

Feature	CFG	PCFG	Neural Parsing
Handles ambiguity	Generates alternatives	Ranks alternatives	Uses learned context
Probability	No	Yes	Yes
Context awareness	Low	Moderate	High
Domain adaptation	Limited	Moderate	Strong
Training requirement	Grammar rules	Probabilities / annotated data	Large training data
Real-world robustness	Low–moderate	Moderate	High

Justification: For modern real-world applications, neural parsing is generally the most effective because it captures contextual and long-distance relationships and handles complex ambiguity. PCFG is useful when interpretability and probabilistic grammar rules are important, while CFG is useful for controlled or rule-based environments. A practical industrial system can combine neural parsing with linguistic constraints or dependency/constituency information.

4. Feature Structures vs Subcategorization Frames

Feature structures represent grammatical properties such as Person, Number, Gender, Tense, Case, and Agreement.

Example:
Subject: Number = Singular, Person = Third
Verb: Number = Singular, Person = Third

This allows “The student reads” and rejects incompatible subject–verb combinations.

Subcategorization frames describe the arguments that a verb requires or permits. For example:

eat → Subject + Object
give → Subject + Object + Recipient sleep
→ Subject

For “The teacher gave the student a book,” the verb gave expects an agent, recipient, and object.

Feature	Feature Structures	Subcategorization Frames
Main purpose	Grammatical constraints	Verb argument requirements
Agreement	Excellent	Limited
Number/person	Yes	Not the main focus
Verb complements	Limited	Excellent
Subject–verb agreement	Strong	Weak
Semantic argument structure	Moderate	Strong
Grammatical validation	Strong	Strong for verb complements

Justification: Neither completely replaces the other. Feature structures are better for enforcing grammatical correctness, especially subject–verb agreement, tense, number, person, and case. Subcategorization frames are better for ensuring that verbs receive the correct arguments and complements.

Best solution: **Feature Structures + Subcategorization Frames → Grammatical + Argument Structure Validation.**

5. Transition-Based Parsing vs Graph-Based Parsing

Transition-based parsing constructs a dependency tree step by step. Typical transitions include SHIFT, REDUCE, LEFT-ARC, and RIGHT-ARC. Its main advantage is speed because decisions are made incrementally.

Graph-based parsing considers possible dependency relationships and searches for the highest-scoring dependency tree.

It can use more global information than a transition-based parser.

Feature	Transition-Based	Graph-Based
Decision-making	Local, step-by-step	Global
Parsing strategy	Sequence of transitions	Search for best tree
Speed	Usually very fast	Usually slower
Memory	Relatively low	Relatively higher
Global information	Limited	Strong
Error propagation	More susceptible	Generally less sequential propagation
Real-time applications	Excellent	Moderate
Large-scale processing	Excellent	More computationally expensive
Accuracy	Very good with modern models	Strong with global scoring

Performance Analysis: Transition-based parsing is generally faster because it makes local decisions and processes each sentence incrementally. It is attractive for search engines, chatbots, voice assistants, real-time NLP, and large-scale data processing. However, an early incorrect transition can affect later decisions.

Graph-based parsing evaluates dependency structures more globally. This can provide strong structural accuracy, but searching over possible structures generally requires more computational resources.

Justification: For a large-scale NLP system requiring very fast dependency parsing, transition-based parsing is generally the better choice, especially when low latency and high throughput are priorities. If global structural accuracy is more important than speed, graph-based parsing may be preferable.

A modern industrial solution can combine a neural encoder with a transition-based parser for a good balance of speed, scalability, and accuracy.

Overall Conclusion

There is no single parsing method that is best for every situation. **CFG** is best for constituent and phrase structure; **dependency parsing** is best for direct word relationships; **Earley parsing** is strong for ambiguous and incomplete dynamic input; **PCFG** is useful for probabilistic ambiguity resolution; **neural parsing** is generally strongest for complex real-world language; **feature structures** are strong for grammatical agreement; **subcategorization frames** are strong for verb argument structures; **transition-based parsing** is best when speed and scalability are critical; and **graph-based parsing** is useful when global structural accuracy is more important.